Combinator Calculus: Decidable Type Checking in Extensional Type Theory

Lexi Aiello[1] and Eric Klavins[2]

[1]Department of Philosophy, University of Washington
[2]Electrical & Computer Engineering Department, University of Washington

November 5, 2026

Abstract

We present the Dependent Combinator Calculus (DCC): an extension of the simply-typed categorical combinators [5] to affine logic and dependent types. DCC has computable extensional equality reflection and decidable type-checking. The calculus is combinatory and has no contexts and no variables. It is similar in expressivity to the Calculus of Constructions on the λ-cube. We demonstrate that the calculus is consistent and strongly normalizing, in addition to the fundamental theorem: equality reflection. We elucidate that computable equality reflection and decidable type-checking can coincide in affine combinatory dependent type theory, and present DCC as evidence. We also present a procedure for compiling the subset of Lean 4 that maps to the base Calculus of Constructions to the calculus. DCC is formalized fully in the Lean 4 proof assistant.

1 Introduction

1.1 Results

- In 2, we present the dependent combinator calculus (**DCC**): our extension of the categorical combinators [4] [5] to extensional dependent type theory and affine logic. We have no variables and no contexts. Types are terms in DCC, as in the Calculus of Constructions with respect to the λ-cube [3]. We describe our β-reduction rules and inference rules.
- In 4.1, we prove some basic properties of conversion in DCC: strong normalization, confluence, uniqueness of normal forms, and decidability of β-equality.
- In 5.1, we prove that type checking is decidable in DCC.
- In 5.2, we prove that subject-reduction and progress hold in DCC, and that DCC is consistent.
- In 6, we prove that extensional equality reflection holds in DCC.The proof is a computable definition, rather than a theorem in **Prop**.
- In **??**, we present a procedure for compiling the subset of Lean 4 that maps to the base Calculus of Constructions to DCC.

All of our results have been formalized in the Lean 4 proof assistant. To this date, DCC is the only calculus with extensional equality reflection and decidable type-checking. However, due to our choice of affine logic and categorical combinators, we inherit these features by construction.

1.2 Background

1.3 The SK Combinator Calculus

Previous attempts by others at combinatory dependent type theory have used the **SK** combinator calculus [2, p. 10:14]. The SK combinators are a Turing-complete term-writing system equivalent to the λ-calculus [1] with the rewrite rules $Kxy = x$ and $Sxyz = xz(yz)$. Good progress has been made, but past

approaches were limited by the minimalism of SK. Types become very large and difficult to work with, relying on a hierarchy of helper ("meta" combinators) to encode basic types [10:17 2, para. 2], and type parameters in the primitive combinators S and K were also quantified externally, rather than internally [10:17 2, para. 2].

By contrast, DCC is based on an extension of categorical combinators (**CCL**) to dependent types and affine logic. In DCC, type arguments are internalized, rather than externally quantified and all universe levels are inferred. No hierarchy of helper ("meta" combinators) are required to support dependent types (e.g., $\Pi[Yx][Zx]$ in [10:17 2, lines 5-7]). The only constructions required are Π-types, Σ-types, identity types, sorts, and the core computational combinators as in CCL.

1.4 Categorical Combinators (CCL)

Categorical combinators [4] [5] are a combinatory logic based on cartesian-closed categories (**CCC**) [12]. They are equivalent to the λ-calculus [p. 205 5, theorem 2.20]. CCL has the following rewrite rules:

$$\frac{}{Fst(x,y) \to_{\beta\text{CCL}} x}\ \text{Fst} \qquad \frac{}{Snd(x,y) \to_{\beta\text{CCL}} x}\ \text{Snd} \qquad \frac{}{(x \circ y)z \to_{\beta\text{CCL}} x(yz)}\ \text{Ass} \qquad \frac{}{Idx \to_{\beta\text{CCL}} x}\ \text{Id}$$

$$\frac{}{\langle x,y\rangle x \to_{\beta\text{CCL}} (Mx,Nx)}\ \text{Pair} \qquad \frac{}{App(x,y) \to_{\beta\text{CCL}} xy}\ \text{App} \qquad \frac{}{\Lambda(x)yz \to_{\beta\text{CCL}} x(y,z)}\ \text{Curry}$$

Cartesian-closed categories are known to be a categorical semantics for the λ-calculus [11]. The equivalence between CCL and the λ-calculus [p. 205 5, theorem 2.20] is due to the shared correspondence [4, p. 6] between CCL, cartesian-closed categories, and the λ-calculus with De Bruijn indices ([6]):

- Composition in CCC corresponds to the composition combinator in CCL.
- Identity morphisms in CCC correspond to the Id combinator in CCL.
- De Bruijn indices correspond to binary projection in CCC (or "couple" projection in CCL) where $[\![0]\!] \coloneqq Snd$ and $[\![n+1]\!] \coloneqq [\![n]\!] \circ Fst$.
- Currying in CCC corresponds to the Λ combinator in CCL [p. 176 4, line 5], such that $[\![\lambda.M]\!] \coloneqq \Lambda([\![M]\!])$.
- Evaluation in CCC corresponds to the App combinator in CCL, such that $[\![MN]\!] \coloneqq App \circ \langle [\![M]\!], [\![N]\!]\rangle$.

The correspondence between CCL and the De Bruijn-indexed λ-calculus makes them a more intuitive compilation target compared to SK, producing more legible terms. To illustrate, we contrast the compilation of church numerals zero and one from λ-calculus with De Bruijn indices to CCL and SK:

$$\begin{array}{lll} 0_\lambda \coloneqq \lambda.\lambda.0 & [\![0_\lambda]\!] = \Lambda(Snd) & [\![0_\lambda]\!]_{\text{SK}} = K(SKK) \\ 1_\lambda \coloneqq \lambda.\lambda.10 & [\![1_\lambda]\!] = \Lambda(App \circ \langle Fst, Snd\rangle) & [\![1_\lambda]\!]_{\text{SK}} = S(S(KS)(S(KK)(SKK)))(K(SKK)) \end{array}$$

The reductions of $1_\lambda xy$, $[\![1_\lambda]\!]xy$, and $[\![1_\lambda]\!]_{\text{SK}}xy$ are as such:

$$\begin{aligned} 1_\lambda xy &= (\lambda.\lambda.10)xy \\ &= (\lambda.x0)y \\ &= xy \end{aligned} \qquad \begin{aligned} [\![1_\lambda]\!]xy &= \Lambda(App \circ \langle Fst, Snd\rangle)xy \\ &= App \circ \langle Fst, Snd\rangle(x,y) \\ &= App(x,y) \\ &= xy \end{aligned}$$

$$\begin{aligned} [\![1_\lambda]\!]_{\text{SK}}xy &= S(S(KS)(S(KK)(SKK)))(K(SKK))xy \\ &= S(KS)(S(KK)(SKK))x(K(SKK)x)y \\ &= KSx(S(KK)(SKK)x)(K(SKK)x)y \\ &= S(KK)(SKK)xy(K(SKK)xy) \\ &= K(SKKx)y(SKKy) \\ &= xy \end{aligned}$$

Despite their notable advantages in term legibility, more closely resembling De Bruijn-indexed λ-calculus, adapting categorical combinators to dependent types has not been attempted thus far.

1.4.1 The Simply-Typed Categorical Combinators

A simply-typed variant ($\mathbf{CCL_K}$) of the categorical combinators is presented in [5, p. 220]. Type arguments are given as points. Type arguments can be inferred altogether in Curry, Pair, Comp, and in couples. The inference rules are as such:

$$\textsc{Fst}\ \frac{}{Fst(A,B) : A \times B \to A} \qquad \textsc{Snd}\ \frac{}{Snd(A,B) : A \times B \to B} \qquad \textsc{Id}\ \frac{}{Id(A) : A \to A}$$

$$\textsc{App}\ \frac{}{App(A,B) : ((A \to B) \times A) \to B} \qquad \textsc{Curry}\ \frac{x : A \times B \to C}{\Lambda(x) : A \to B \to C} \qquad \textsc{Pair}\ \frac{x : A \to B, y : A \to C}{\langle x, y \rangle : A \to B \times C}$$

$$\textsc{Comp}\ \frac{x : B \to C, y : A \to B}{x \circ y : A \to C} \qquad \textsc{Coup}\ \frac{x : A, y : B}{(x, y) : A \times B}$$

1.5 Affine Logic

Affine logic refers to any logic where duplication ("contraction") is restricted or prohibited, although such logics are usually related to linear logic [p. 173 10, sec. 1, par. 1]. Linear logic is a constructive logic [8] wherein terms in context must be used exactly once, except in the "exponential" modalities (! and ?) [8, p. 3]:

$$\otimes\ \frac{\vdash A, C \qquad \vdash B, D}{\vdash A \otimes B, C, D} \qquad \parr\ \frac{\vdash A, B, C}{\vdash A \parr B, C} \qquad 1\oplus\ \frac{\vdash A, C}{\vdash A \oplus B, C} \qquad 2\oplus\ \frac{\vdash B, C}{\vdash A \oplus B, C} \qquad \&\ \frac{\vdash A, C \qquad \vdash B, C}{\vdash A \& B, C}$$

$$W?\ \frac{\vdash B}{\vdash ?A; B} \qquad C?\ \frac{\vdash ?A, ?A, B}{\vdash ?A, B} \qquad D?\ \frac{\vdash A, B}{\vdash ?A, B} \qquad !,\ \frac{\vdash A, ?B}{\vdash !A, ?B}$$

Weakning is retained in affine logic. Most importantly, affine logic forms a decidable fragment of first-order logic [9].

2 The Dependent Combinator Calculus

The Dependent Combinator Calculus (DCC) is a dependent type theory where types are terms which has no contexts and no variables (it is combinatory). It is based on an extension of the simply-typed categorical combinators to dependent types. DCC uses affine logic, meaning term duplication is restricted in such a way that all programs are provably terminating by syntactic size, even if they are not well-typed. We use this fact to show that DCC is strongly-normalizing, has decidable type-checking, is consistent, and that equality reflection holds as a computable metatheoretic definition.

3 Term Syntax

The base objects are a modified superset of the base terms in $\mathrm{CCL_K}$. Products are replaced by Σ, arrows by Π, sorts are added, and bottom, top, and boolean types are added with the corresponding introducing terms. The syntax is given in S-expressions for clarity. A term (E) is either a constant of term arity zero (E_c), a parenthesized combinator (C) juxtaposed with its points, or a parenthesized application of two terms.

Definition 1. *Constants (E_c) are sort levels (written as Sort u), the bottom ($\mathbb{0}$), top ($\mathbb{1}$) type and its inhabitant ($*$), the boolean type ($\mathbb{2}$) along with its inhabitants $\mathbb{1}_2$, and $\mathbb{0}_2$, and an opaque "hole" term used internally in type-checking:*

$$E_c \coloneqq \{Sort\ u, \mathbb{0}, \mathbb{1}, \mathbb{2}, *, \mathbb{1}_2, \mathbb{0}_2, hole(n)\}$$

where $u, n \in \mathbb{N}$.

Definition 2. *Combinators are identified by the set of names* C*:*

$C \coloneqq \{\mathbb{0}.Rec, \mathbb{2}.Rec, ! <>, \$ <>, *, !, \&, =, Eval, Eql, =', refl, `, Id, Fst, Snd, Swap, \Sigma, \Sigma', \Lambda, !\Lambda, \Pi, \circ, \langle\,\rangle, , \}$

Definition 3. *A term* (E) *is either a constant* E_c*:*

$$E_c \subset E$$

or a parenthesized combinator C *juxtaposed by its points which are also terms:*

$$\{(c\ x_0 \dots x_n) \mid c \in C, \{x \mid x \in x_0 \dots x_n\} \subset E\} \ \subset E$$

or a parenthesized application of a list of juxtaposed terms:

$$\{(f\ x_0 \dots x_n) \mid f \in E, \{x \mid x \in x_0 \dots x_n\} \subset E\} \subset E$$

Example 1. *If* $A \in E$*, then* $(Id\ A) \in E$.

Example 2. *If* $A, x \in E$*, then* $((Id\ A)x) \in E$.

Example 3. *If* $u \in \mathbb{N}$*, then Sort* $u \in E$.

Example 4. *If* $B, x, y \in E$*, then* $(, B\ x\ y) \in E$.

4 Conversion Rules

Conversion uses call-by-value operational semantics and is a modified superset of those in CCL. Terms are reduced in any particular order of: head reductions, points of the term if it is a combinator, or the constituent function and argument if it is an application. We have global confluence, strong normalization, and uniqueness of normal forms. We do not have a recursor for equality, since DCC is extensional 6 and propositional equality coincides with definitional equality, so it is not necessary. When $A, B \in E$, we use $A \to_\beta B$ to denote single-step β-reduction, $A \to_\beta^* B$ to denote star-step β-reduction, and $A \to_\beta^! B$ to denote β-reduction to a normal form.

Definition 4. *The identity, quotation, and composition are the same as in* CCL_K*:*

$$\textsc{Id}\ \frac{}{((Id\ A)x) \to_\beta x} \qquad \textsc{Quot}\ \frac{}{((`B\ x)y) \to_\beta x} \qquad \textsc{Comp}\ \frac{}{((\circ\ x\ y)z) \to_\beta (x(yz))}$$

Definition 5. *Projection over couples and currying are the same as in CCL, but with type arguments added:*

$$\textsc{Fst}\ \frac{}{((Fst\ B)(, B_2\ x\ y)) \to_\beta x} \qquad \textsc{Snd}\ \frac{}{((Snd\ B)(, B_2\ x\ y)) \to_\beta y} \qquad \textsc{Curry}\ \frac{}{((\Lambda\ A\ B\ x)yz) \to_\beta x(, B\ y\ z)}$$

Definition 6. *Evaluation of couples is given as the usual recursor for* Σ*:*

$$\textsc{Eval}\ \frac{}{((Eval\ A\ B\ x)(, B_2\ y\ z)) \to_\beta (xyz)}$$

Definition 7. *Exchange is permitted on nondependent couples as an optimization, yielding a couple quoting the first component:*

$$\textsc{Swap}\ \frac{}{((Swap\ A\ B)(, B_2\ x\ y)) \to_\beta ((`B\ A)\ y\ x)}$$

Definition 8. *Boxed terms may be unboxed, mapped into banged types, and mapped over into other boxed terms:*

$$\textsc{Unbox}\ \frac{}{((*\ A)(\&\ x)) \to_\beta x} \qquad \textsc{Map}\ \frac{}{((!<>\ x)(\&\ y)) \to_\beta (!\ (xy))} \qquad \textsc{Fmap}\ \frac{}{((\$<>\ x)(\&\ y)) \to_\beta (\&\ (xy))}$$

Definition 9. *Boxed terms can also be curried:*

$$\textsc{Curry'}\quad \frac{}{((!\Lambda\ A\ B\ x)(\&\ y)(\&\ z)) \to_\beta (x(\&\ (,B\ y\ z)))}$$

Definition 10. *As opposed to its usual behavior in CCL, the pair rule consumes boxed terms and duplicates its argument so as to be fully dependent:*

$$\textsc{Pair}\quad \frac{}{((\langle,\rangle B\ x\ y)(\&\ z)) \to_\beta (,(Bz)\ (xz)\ (yz(xz)))}$$

Definition 11. *Equality types can also be formed by the combinators $='$ and Eql, which index an environment determining the arguments' types:*

$$\textsc{Eq'}\quad \frac{}{((='\ B)xyz) \to_\beta (=(Bx)\ y\ z)} \qquad \textsc{Eql}\quad \frac{}{((Eql\ B)(\&\ x)(\&\ y)) \to_\beta (=(Bx)\ y\ y)}$$

Definition 12. *Σ-types can be formed from boxes:*

$$\textsc{Sigma'}\quad \frac{}{((\Sigma'\ B\ C)(\&\ x)) \to_\beta (\Sigma\ (Bx)\ (Cx))}$$

Definition 13. *We have a recursor for $\mathbb{2}$:*

$$\textsc{Bool Recl.}\quad \frac{}{((\mathbb{2}.Rec\ B\ x\ y)z\mathbb{1}_2) \to_\beta (xz\mathbb{1}_2)} \qquad \textsc{Bool Recr.}\quad \frac{}{((\mathbb{2}.Rec\ B\ x\ y)z\mathbb{0}_2) \to_\beta (yz\mathbb{0}_2)}$$

Definition 14. *Finally, there are pointwise congruences for combinators and congruences for application:*

$$\textsc{Point}\quad \frac{c \in C \qquad i < n \qquad x_i \to_\beta x_i'}{(c\ x_0 \dots x_n) \to_\beta (c\ x_0' \dots x_n')} \qquad \textsc{Congr. Fun.}\quad \frac{f \to_\beta f'}{(fx) \to_\beta (f'x)} \qquad \textsc{Congr. Arg.}\quad \frac{x \to_\beta x'}{(fx) \to_\beta (fx')}$$

4.1 Some Properties of Conversion

DCC is provably strongly normalizing due to our choice of affine logic. DCC is also locally confluent, and thus globally confluent by Newmann's lemma. As such, normal forms are unique and β-equality is decidable. We use this fact to establish judgmental decidability.

4.1.1 Strong Normalization by Decreasing Measure

We show strong normalization by defining a function $size : E \to \mathbb{N}$ such that $A \to_\beta B$ implies $size(B) < size(A)$.

Definition 15. *If $e \in E_c$, then $size(e) = 1$, and if $(fx) \in E$ and (fx) is an application, then $size((fx)) = size(f) + size(x)$. Otherwise, combinator sizes are defined as:*

$$\begin{aligned}
size((Eql\ B)) &= size(B) + 2 & size((='\ B)) &= size(B) + 2\\
size((Id\ A)) &= size(A) + 1 & size((refl\ B)) &= size(B) + 1\\
size((\Sigma\ A\ B)) &= size(A) + size(B) + 1 & size((\Sigma'\ B\ C)) &= size(B) + size(C) + 1\\
size((`B\ x)) &= size(B) + size(x) + 1 & size((,B\ x\ y)) &= size(B) + size(x) + size(y) + 1\\
size((Fst\ B)) &= size(B) + 1 & size((Snd\ B)) &= size(B) + 1\\
size((Swap\ B\ C)) &= size(B) + size(C) + 2 & size((\circ\ x\ y)) &= size(x) + size(y) + 1\\
size((!\ x)) &= size(x) + 1 & size((*\ A)) &= size(A) + 1\\
size((\&\ x)) &= size(x) * 5 + 1 & size((!<>\ x)) &= size(x) + 2\\
size((\Pi\ B\ C)) &= size(B) + size(C) + 1 & size(($<>\ x)) &= size(x) * 5 + 1
\end{aligned}$$

$$
\begin{aligned}
size((=\ A\ x\ y)) &= size(A) + size(x) + size(y) + 1 \\
size((2.Rec\ x\ y\ z)) &= size(x) + size(y) + size(z) + 1 \\
size((\mathbb{0}.Rec\ x)) &= size(x) + 1 \\
size((\langle,\rangle B\ x\ y)) &= 3 * (size(B) + (2 + size(x)) + size(y) + 3) \\
size((!\Lambda\ A\ B\ x)) &= 2 * size(A) + 6 * (size(B) + 1) + 2 * size(x) + 5 \\
size((\Lambda\ A\ B\ x)) &= size(A) + size(B) + size(x) + 2 \\
size((Eval\ A\ B\ x)) &= size(A) + size(B) + size(x) + 1
\end{aligned}
$$

Lemma 1. *If $c \in C$ and $\{x_0 \dots x_n\} \subset E$, then $size(x_i) < size((c\ x_0 \dots x_n))$ for all $i < n$.*

Proof. By induction on the structure of expressions. □

Theorem 1. *Strong-normalization: If $A, B \in E$ and $A \to_\beta B$, then $size(B) < size(A)$.*

Proof. We proceed by induction on the reduction relation.

Id: $size(((Id\ A)x)) = size(A) + 1 + size(x) > size(x)$

Quot: $size(((`B\ x)y)) = size(B) + size(x) + 1 + size(y) > size(x)$

Comp: $size(((\circ\ x\ y)z)) = size(x) + size(y) + 1 + size(z) > size(x) + size(y) + size(z)$

Fst: $size(((Fst\ B)(, B_2\ x\ y))) = size(B) + 1 + size(B_2) + size(x) + size(y) + 1 > size(x)$

Snd: $size(((Snd\ B)(, B_2\ x\ y))) = size(B) + 1 + size(B_2) + size(x) + size(y) + 1 > size(y)$

Curry:

$$
\begin{aligned}
size(((\Lambda\ A\ B\ x)yz)) &= size(A) + size(B) + size(x) + 2 + size(y) + size(z) \\
&> size(x) + size(B) + size(y) + size(z)
\end{aligned}
$$

Eval:

$$
\begin{aligned}
size(((Eval\ A\ B\ x)(, B_2\ y\ z))) &= size(A) + size(B) + size(x) + 1 + size(B_2) + size(y) + size(z) + 1 \\
&> size(x) + size(y) + size(z)
\end{aligned}
$$

Swap:

$$
\begin{aligned}
size(((Swap\ A\ B)(, B_2\ x\ y))) &= size(B) + size(C) + 2 + size(B_2) + size(x) + size(y) + 1 \\
&> size(B) + size(A) + size(y) + size(x) + 1
\end{aligned}
$$

Unbox: $size(((*\ A)(\&\ x))) = size(A) + 1 + (size(x) * 5 + 1) > size(x)$

Map: $size(((!<>\ x)(\&\ y))) = size(x) + 2 + (size(y) * 5 + 1) > size((xy)) + 1$

Fmap: $size(((\$<>\ x)(\&\ y))) = (size(x) * 5 + 1) + (size(y) * 5 + 1) > size((xy) * 5 + 1)$

Curry':

$$
\begin{aligned}
size(((!\Lambda\ A\ B\ x)(\&\ y)(\&\ z))) &= 2 * size(A) + 6 * (size(B) + 1) \\
&\quad + 2 * size(x) + 5 + size(y) * 5 + 1 + size(y) * z + 1 \\
&> size(x) + ((5 * (size(B) + size(y) + size(z))) + 1)
\end{aligned}
$$

Pair:

$$\begin{aligned} size((((\langle,\rangle B\ x\ y)(\&\ z))) &= 3 * (size(B) + (size(x) + 2) + size(y) + 3) + (5 * size(z) + 1) \\ &> 5 * (size(B) + 4 * size(z) + 2 * size(x) + size(y)) + 1 \end{aligned}$$

Eq':

$$\begin{aligned} size(((='\ \ B)xyz)) &= size(A) + 2 + size(x) + size(y) + size(z) \\ &> (size(B) + size(x)) + size(y) + size(z) + 1 \end{aligned}$$

Eql:

$$\begin{aligned} size(((Eql\ B)(\&\ x)(\&\ y))) &= size(B) + 2 + (size(x) * 5 + 1)(size(y) * 5 + 1) \\ &> (size(B) + size(x)) + size(y) + size(y) + 1 \end{aligned}$$

Sigma':

$$\begin{aligned} size(((\Sigma'\ B\ C)(\&\ x))) &= size(B) + size(C) + 1 + (size(x) * 5 + 1) \\ &> (size(B) + size(x)) + (size(C) + size(x)) + 1 \end{aligned}$$

Bool Recl.:

$$\begin{aligned} size(((\mathbb{2}.Rec\ B\ x\ y)z\mathbb{1}_{\mathbb{2}})) &= size(B) + size(x) + size(y) + 1 + size(z) \\ &> size(x) + size(z) + size(\mathbb{1}_{\mathbb{2}}) \end{aligned}$$

Bool Recr.:

$$\begin{aligned} size(((\mathbb{2}.Rec\ B\ x\ y)z\mathbb{1}_{\mathbb{2}})) &= size(B) + size(x) + size(y) + 1 + size(z) \\ &> size(y) + size(z) + size(\mathbb{0}_{\mathbb{2}}) \end{aligned}$$

Point: If $(c\ x_0 \dots x_n) \in E$ and $i < n$, then $size(x_i) < size((c\ x_0 \dots x_n))$ 1. And if $x_i \to_\beta x_i'$, then, by the inductive hypothesis, $size(x_i') < size(x_i)$, so $size((c\ x_0 \dots x_n)) > size((c\ x_0' \dots x_n'))$.

Congr. Fun.: If $(fx) \in E$ and $f \to_\beta f'$, then, by the inductive hypothesis, $size(f') < size(f)$, and $size((fx)) = size(f) + size(x) > (size((f'x)) = size(f') + size(x))$.

Congr. Arg.: If $(fx) \in E$ and $x \to_\beta x'$, then, by the inductive hypothesis, $size(x') < size(x)$, and $size((fx)) = size(f) + size(x) > (size((fx')) = size(f) + size(x'))$.

✔ [Formalized: `Expr.Red.decreasing`] □

Lemma 2. *If $A, B \in E$ and $A \to_\beta^* B$, then $size(B) \leq size(A)$.*

Proof. Suppose $A = B$. Then, $size(A) = size(B)$, so $size(B) \leq size(A)$. If $A \neq B$, then there exists $B_0 \in E$ such that $A \to_\beta B_0$ and $B_0 \to_\beta^* B$. So, by 1, $size(B_0) < size(A)$, and by the inductive hypothesis, $size(B) \leq size(B_0)$, so $size(B) \leq size(A)$.
✔ [Formalized: `Expr.decreasing?`] □

Lemma 3. *Point congruences are closed under star-step reduction: If $c \in C$ and $\{x_0 \dots x_n\} \subset E$ and $i < n$ and $x_i \to_\beta^* x_i'$, then $(c\ x_0 \dots x_n) \to_\beta^* (c\ x_0' \dots x_n')$.*

Proof. Suppose $h \vdash x_i \to_\beta^* x_i'$. Then, if $x_i = x_i'$, then $(c\ x_0 \dots x_n) \to_\beta^* (c\ x_0' \dots x_n')$ by reflexivity. If $x_i \neq x_i'$, then there must exist $x_{0,i}'$ such that $h_0 \vdash x_i \to_\beta x_{0,i}'$ and $h_1 \vdash x_{0,i}' \to_\beta^* x_i'$. And, by 1, $size(x_{0,i}') < size(x_i)$. So, by induction, $(c\ x_{0,0} \dots x_{0,n}) \to_\beta^* (c\ x_0' \dots x_n')$. Hence, $(c\ x_0 \dots x_n) \to_\beta^* (c\ x_0' \dots x_n')$ by transitivity of single-step application of the point rule with respect to h_0 and h_1.
✔ [Formalized: `Expr.aff_mk_point'`] □

Lemma 4. *Function congruence is closed under star-step reduction: If $(fx) \in E$ and $f \to_\beta^* f'$, then $(fx) \to_\beta^* (f'x)$.*

Proof. The proof proceeds similarly as in 3, but with the function congruence single-step rule.
✔ [Formalized: `Expr.congrFun'`] □

Lemma 5. *Argument congruence is closed under star-step reduction: If $(fx) \in E$ and $x \to_\beta^* x'$, then $(fx) \to_\beta^* (fx')$.*

Proof. The proof proceeds similarly as in 3, but with the argument congruence single-step rule.
✔ [Formalized: `Expr.congrArg'`] □

Theorem 2. *Local confluence: If $A, B, C \in E$ and $A \to_\beta B$ and $A \to_\beta C$, then there exists $D \in E$ such that $B \to_\beta^* D$ and $C \to_\beta^* D$.*

Proof. The proof proceeds by induction on $A \to_\beta B$ and parallel case analysis on $A \to_\beta C$. We handle three possible cases: head-reduction where B and C are the head-reducts of A; point-reduction where A is not an application but a combinator with points; congruences where A is of the form $(fx) \in E$ and where either B or C are of the form $(f'x)$ where $f \to_\beta f'$ or (fx') where $x \to_\beta x'$.

In the head step cases where $f, x \in E$ and $A = (fx) \in E$, the step $A \to_\beta B$ or $A \to_\beta C$, respectively, is destructed until atomic terms which are not combinators with points or further applications are met. Then, a step is reconstructed for some $D \in E$ such that $B \to_\beta D$ and $C \to_\beta D$. In the case where $A = (c\ x_0 \dots x_n) \in E$ (i.e., where A is a term with points), the step $A \to_\beta B$ or $A \to_\beta C$ is destructed, respectively, into its atomic point steps and a step is reconstructed similarly. In the case where $A = (fx) \in E$ and f is not obviously a combinator, we case split on f and perform the same steps as in the head reduction case until an atomic f is reached, such that, by the inductive hypothesis and the function congruence rule, we may reconstruct a step. We do similarly for x and handle all point-term cases until an atomic x is reached, in which case the inductive hypothesis applies.

To illustrate, suppose $((Id\ A)x) \in E$ and $h_1 \vdash ((Id\ A)x) \to_\beta B$ and $h_2 \vdash ((Id\ A)x) \to_\beta C$:

Function congruence in h_1 on $(Id\ A)$: Suppose B is of the form $(f'x)$ where $(Id\ A) \to_\beta f'$. Then, f' must be constructed by the point rule (14), so there exists A' such that $A \to_\beta A'$. We proceed by case-splitting on h_2:

- **Function congruence in h_2 on $(Id\ A)$:** Suppose C is of the form $(f_2'x)$ where $(Id\ A) \to_\beta f_2'$. Then, f_2' must be constructed by the point rule (14), so there exists A_2' such that $A \to_\beta A_2'$. So, by the inductive hypothesis, there exists $A'' \in E$ such that $A' \to_\beta^* A''$ and $A_2' \to_\beta^* A''$. And, by 3, $(Id\ A') \to_\beta^* (Id\ A'')$ and $(Id\ A_2') \to_\beta^* (Id\ A'')$. So, by 4, $((Id\ A')x) \to_\beta^* ((Id\ A'')x)$ and $((Id\ A_2')x) \to_\beta^* ((Id\ A'')x)$.
- **Argument congruence in h_2 on x:** Suppose C is of the form $((Id\ A)x')$ where $x \to_\beta x'$. Then, by the single-step rule Id 4, $((Id\ A')x) \to_\beta x \to_\beta x'$ and $((Id\ A)x) \to_\beta ((Id\ A)x') \to_\beta x'$, so $B \to_\beta^* x'$ and $C \to_\beta^* x'$.

Argument congruence in h_1 on x: Suppose B is of the form $((Id\ A)x')$ where $x \to_\beta x'$. Then, the proof is as above, except h_1 and h_2 are inverted.

Head reduction in h_1 on $((Id\ A)x)$: Suppose $B = x$. Then, if h_2 is a function congruence such that $(Id\ A) \to_\beta f'$, f' must be of the form $(Id\ A')$ where $A \to_\beta A'$, so $B \to_\beta x$ and $C \to_\beta x$. If h_2 is an argument congruence such that $h \vdash x \to_\beta x'$, then $B \to_\beta x'$ by h and $C \to_\beta x'$ by reflexivity.

✔ [Formalized: `Expr.Red.conf`] □

Theorem 3. *Global confluence: If $A, B, C \in E$ and $A \to_\beta^* B$ and $A \to_\beta^* C$, then there exists $D \in E$ such that $B \to_\beta^* D$ and $C \to_\beta^* D$.*

Proof. The proof proceeds by Newmann's lemma:

$A = B$ and $A = C$: By reflexivity, $B \to_\beta^* B$ and $C \to_\beta^* B$.

$h \vdash A = B$ but $A \neq C$: By h, $B \to_\beta^* C$, and by reflexivity, $C \to_\beta^* C$.

$A \neq B$ and $h \vdash A = C$: By h, $C \to_\beta^* B$, and by reflexivity, $B \to_\beta^* B$.

$A \neq B$ and $A \neq C$: Then, there exist $B_0, C_0 \in E$ such that $A \to_\beta B_0$ and $A \to_\beta C_0$ and $B_0 \to_\beta^* B$ and $C_0 \to_\beta^* C$, so, by 2, there exists $D \in E$ such that $B_0 \to_\beta^* D$ and $C_0 \to_\beta^* D$. And, due to 1 and 2, $size(B_0) < size(A)$ and $size(C_0) < size(A)$. So, by well-founded induction given $B_0 \to_\beta^* B$, $B_0 \to_\beta^* D$, $C_0 \to_\beta^* C$, and $C_0 \to_\beta^* D$, there exist $B_2, C_2 \in E$ such that $B \to_\beta^* B_2$ and $D \to_\beta^* B_2$ and $C \to_\beta^* C_2$ and $D \to_\beta^* C_2$. And, similarly, since $B \to_\beta^* B_2$ and $B \to_\beta^* C_2$ (via D), there exists $D' \in E$ such that $B_2 \to_\beta^* D'$ and $C_2 \to_\beta^* D'$, so $B \to_\beta^* D'$ and $C \to_\beta^* D'$.

✔ [Formalized: `Expr.RedRel.globally_confluent`] □

4.2 Decision Procedure for β-Equality

Our β-reduction rules are implemented as a total, terminating β-reduction engine. Reduction proceeds by repeated single-step reduction via a function $step? : (A : E) \to Option(\Sigma'(B : E), A \to_\beta B)$ and well-founded recursion by decreasing syntactic size 1. The reducer has has soundness and completeness with respect to $\beta^!$. The reducer is used to decide β-equality and is central to decidable type-checking.

In applications $(fx) \in E$, the *step* attempts a head single-step, if it is available:

```
@[dcc_force_reduce, dcc_force_aff, aesop unsafe norm (rule_sets := [DccForceReduce])]
def step_head? (e : Expr) : Option ((e' : Expr) × (PULift.{1, 0} (Red e e'))) :=
  match e with
  | (DCC? ((Σ' _B _C) (& _x))) => .some ⟨_, ⟨.sigma⟩⟩
  | (DCC? (((?.Rec _motive _x _y) _env ?₂)) =>
    .some ⟨_, ⟨.ind_bool_true⟩⟩
  | (DCC? (((?.Rec _motive _x _y) _env ?₂)) =>
    .some ⟨_, ⟨.ind_bool_false⟩⟩
  | DCC? ((!<> _f) (& _x))                                  => .some ⟨_, ⟨.map⟩⟩
  | DCC? (($<> _f) (& _x))                                  => .some ⟨_, ⟨.fmap⟩⟩
  | DCC? ((* _α) (& _x))                                   => .some ⟨_, ⟨.unbox⟩⟩
  | DCC? ((⟨,⟩_β _M _N) (& _x))                           => .some ⟨_, ⟨.pair⟩⟩
  | (DCC? ((Eval _α _β _x) (,__β _y _z)))                 => .some ⟨_, ⟨.evalc⟩⟩
  | (DCC? ((Eql _f_α) (& _env) (& _y)))                   => .some ⟨_, ⟨.eql⟩⟩
  | (DCC? ((=' _f_α) _x _y _z))                  => .some ⟨_, ⟨.eq_mk⟩⟩
  | (DCC? ((`_β _x) _y))                                  => .some ⟨_, ⟨.quot⟩⟩
  | (DCC? ((Id _α) _x))                                    => .some ⟨_, ⟨.id⟩⟩
  | (DCC? ((Fst _α) (,_γ _a _b)))                          => .some ⟨_, ⟨.fstc⟩⟩
  | (DCC? ((Swap _B _C) (,_B₂ _a _b)))                     => .some ⟨_, ⟨.swap⟩⟩
  | (DCC? ((Snd _α) (,_γ _a _b)))                          => .some ⟨_, ⟨.sndc⟩⟩
  | (DCC? ((∘ _x _y) _z))                                  => .some ⟨_, ⟨.comp⟩⟩
  | (DCC? ((!Λ _α _β _x) (& _y) (& _z)))                   => .some ⟨_, ⟨.curr'⟩⟩
  | (DCC? ((Λ _α _β _x) _y _z)) => .some ⟨_, ⟨.curr⟩⟩
  | _ => .none
```

✔ [Formalized: `Expr.step_head?`]

If no head step is available, it attempts to step f, then if f cannot be stepped, it attempts to step x. If e is a combinator of the form $(c\ x_0 \ldots x_n)$ such that $c \in C$ and $\{x_0 \ldots x_n\} \subset E$, the first point x_i where $i < n$ which can make a step is used:

```
@[dcc_force_reduce, dcc_force_aff, aesop unsafe norm (rule_sets := [DccForceReduce])]
def step? (e : Expr) : Option ((e' : Expr) × (PULift.{1, 0} (Red e e'))) :=
  open Dsl in match step_head? e with
  | .some h => .some h
  | .none =>
    match e.cases_point_arity' with
    | .inl ⟨heq⟩ =>
      match e with
      | DCC? (f x) =>
        match f.step?, x.step? with
```

```
        | .some ⟨f', ⟨h₁⟩⟩, _ => .some ⟨(DCC? (f' x)), ⟨.congrFun h₁⟩⟩
        | _, .some ⟨x', ⟨h₁⟩⟩ => .some ⟨(DCC? (f x')), ⟨.congrArg h₁⟩⟩
        | _, _ => .none
      | _ => .none
    | .inr (.inl ⟨heq⟩) =>
      match (e.points.get ⟨0, by simp [heq]⟩).step? with
      | .some ⟨_, ⟨h₁⟩⟩ =>
        .some ⟨_, ⟨.point 0 (by simp [heq]) h₁⟩⟩
      | .none => .none
    | .inr (.inr (.inl ⟨heq⟩)) =>
      match (e.points.get ⟨0, by simp [heq]⟩).step?, (e.points.get ⟨1, by simp [heq]⟩).step?
      ↪ with
      | .some ⟨_, ⟨h₁⟩⟩, _ =>
        .some ⟨_, ⟨.point 0 (by simp [heq]) h₁⟩⟩
      | _, .some ⟨_, ⟨h₂⟩⟩ =>
        .some ⟨_, ⟨.point 1 (by simp [heq]) h₂⟩⟩
      | _, _ => .none
    | .inr (.inr (.inr (.inl ⟨heq⟩))) =>
      match (e.points.get ⟨0, by simp [heq]⟩).step?, (e.points.get ⟨1, by simp [heq]⟩).step?
      ↪ with
      | .some ⟨_, ⟨h₁⟩⟩, _ =>
        .some ⟨_, ⟨.point 0 (by simp [heq]) h₁⟩⟩
      | _, .some ⟨_, ⟨h₂⟩⟩ =>
        .some ⟨_, ⟨.point 1 (by simp [heq]) h₂⟩⟩
      | _, _ =>
        match (e.points.get ⟨2, by simp [heq]⟩).step? with
        | .some ⟨_, ⟨h₃⟩⟩ =>
          .some ⟨_, ⟨.point 2 (by simp [heq]) h₃⟩⟩
        | .none => .none
    | .inr (.inr (.inr (.inr ⟨heq⟩))) =>
      match (e.points.get ⟨0, by simp [heq]⟩).step?, (e.points.get ⟨1, by simp [heq]⟩).step?
      ↪ with
      | .some ⟨_, ⟨h₁⟩⟩, _ =>
        .some ⟨_, ⟨.point 0 (by simp [heq]) h₁⟩⟩
      | _, .some ⟨_, ⟨h₂⟩⟩ =>
        .some ⟨_, ⟨.point 1 (by simp [heq]) h₂⟩⟩
      | _, _ =>
        match (e.points.get ⟨2, by simp [heq]⟩).step?, (e.points.get ⟨3, by simp
        ↪ [heq]⟩).step? with
        | .some ⟨_, ⟨h₃⟩⟩, _ =>
          .some ⟨_, ⟨.point 2 (by simp [heq]) h₃⟩⟩
        | _, .some ⟨_, ⟨h₄⟩⟩ =>
          .some ⟨_, ⟨.point 3 (by simp [heq]) h₄⟩⟩
        | _, _ => .none
termination_by (e.size, sizeOf e)
decreasing_by
  any_goals (grind)
```

✔ [Formalized: `Expr.step?`]

Reduction to a normal form proceeds by repeated single-step reduction and well-founded recursion given $h \vdash A \to_\beta B$ implies $size(B) < size(A)$ until no step is available:

```
@[dcc_force_reduce, aesop norm (rule_sets := [DccForceReduce])]
def reduce.{u} {α : Type u} {t : α → ℕ} (r : α → α → Prop) [hR : RedRel r t]
  (e : α) : ((e' : α) × (PULift.{u + 1, 0} (Relation.ReflTransGen r e e'))) :=
  match hR.run e with
  | .some ⟨e', ⟨h⟩⟩ =>
    have _size_ : t e' < t e := hR.decreasing h
    let ⟨e'', ⟨h₂⟩⟩ := reduce r (t := t) e'
    ⟨e'', ⟨.trans (.single h) h₂⟩⟩
  | .none => ⟨e, ⟨.refl⟩⟩
termination_by t e
```

✔ [Formalized: `Expr.reduce`]

Theorem 4. *Semi-completeness of single-step reduction: If $A, B \in E$ and $h \vdash A \to_\beta B$, then there exists $B' \in E$ and $h' \vdash A \to_\beta B'$ such that $step?A = some\langle B, h'\rangle$.*

Proof. Suppose $h \vdash A \to_\beta B$. The proof proceeds by case-splitting on the definition of *step?*:

***A* can make a head step:** If A can make a head step, then there exists some $B' \in E$ such that $step?A = some\langle B, h\rangle$ and where h is the corresponding single-step rule 4.

A **is of the form** (fx) **where** $f, x \in E$ **and** A **cannot make a head step:** If A cannot make a head step and if f cannot make a head step, then x must make a head step, and there exists $x' \in E$ where $h_x \vdash x \to_\beta x'$ and $step?(fx) = \langle (fx'), congrArgh_x \rangle$. This is demonstrated by case-splitting on h and by the fact that A can make neither a head step nor a step on f. If f can make a head step, then there exists $f' \in E$ such that $h_f \vdash f \to_\beta f'$ and $step?(fx) = \langle (f'x), congrFunh_f \rangle$.

A **is of the form** $(c\ x_0 \dots x_n)$ **where** $c \in C$ **and** $\{x_0 \dots x_n\} \subset E$**:** We repeatedly case-split on each $i < n$ which can or cannot make a step. In each case, we apply the point 14 rule, and if none can make a step, we show a contradiction by case-splitting on h.

✔ [Formalized: `Expr.semi_complete`] □

Theorem 5. *Reducts given by reduce are normal forms: If $A, B \in E$ and $h \vdash A \to_\beta^* B$ and $reduceA = \langle B, h \rangle$, then B is a normal form.*

Proof. The proof proceeds by case-splitting on the definition of *reduce*. If $step?A = none$, then A is a normal form, since otherwise, $step?A \neq none$ by 4. If $step?A = some\langle B_0, h_0 \rangle$ such that $h_0 : A \to_\beta B_0$, then $size(B_0) < size(A)$ by 1. So, by well-founded induction, B_0 is a normal form.
✔ [Formalized: `Expr.reduce_norm`] □

Theorem 6. *Completeness of β-reduction with respect to normal forms: If $A, B \in E$ and $h \vdash A \to_\beta^* B$ and B is a normal form (that is, there does not exist $C \in E$ such that $B \to_\beta C$), then $reduceRedA = \langle B, h \rangle$.*

Proof. This fact is shown by case-splitting on the definition of *reduce*:

$step?A = none$**:** If no step can be made, then h must be proven by reflexivity (that is, $A = B$). Otherwise, a step would be possible, in which case, by 4, $step?A \neq none$. By the definition of *reduce*, $reduceA = \langle B, refl \rangle$.

$step?A = \langle B', h' \rangle$**, where** $h' \vdash A \to_\beta B'$**:** If a step can be made, then by confluence 3, there exists $D \in E$ such that $B' \to_\beta^* D$ and $B \to_\beta^* D$. In case A reaches B in a single step, then $B = D$, since B is a normal form. And, since $size(B') < size(A)$ and $B' \to_\beta B$, well-founded induction applies. That is, there exists h' such that $reduceB' = \langle B, h' \rangle$. So, there exists h'' such that $reduceA = \langle B, h'' \rangle$, and by propositional extensionality, $h = h''$.

✔ [Formalized: `Expr.reduce_complete`] □

Theorem 7. *Uniqueness of normal forms: if $A, B, C \in E$ and $h_1 \vdash A \to_\beta^! B$ and $h_2 \vdash A \to_\beta^! C$, then $B = C$.*

Proof. By completeness of β-reduction 6, $reduceA = \langle B, h_1 \rangle$ and $reduceA = \langle C, h_2 \rangle$, so $B = C$.
✔ [Formalized: `Expr.norm_unique`] □

Using our reduction function with soundness and completeness, we decide β-equality.

Definition 16. *If $A, B \in E$, then $A =_\beta B$ iff there exists $C \in E$ such that $A \to_\beta^! C$ and $B \to_\beta^! C$.*

Theorem 8. *If $A, B \in E$, then $A =_\beta B$ or $A \neq_\beta B$.*

Proof. Let $\langle A_1, h_1 \rangle \coloneqq reduceA$ and $\langle B_1, h_2 \rangle \coloneqq reduceB$. Then, A_1 and B_1 are normal forms by 5. We proceed by cases:

$A_1 = B_1$**:** If $A_1 = B_1$, then $A \to_\beta^! A_1$ and $B \to_\beta^! A_1$, so $A =_\beta B$.

$A_1 \neq B_1$**:** If $A_1 \neq B_1$, then $A \neq_\beta B$, since $A =_\beta B$ implies that there exists $C \in E$ such that $h_3 \vdash A \to_\beta^! C$ and $h_4 \vdash B \to_\beta^! C$. But, then $reduceA = \langle C, h_3 \rangle = \langle C, h_4 \rangle = reduceB$. Hence, a contradiction is derived.

□

5 Inference Rules

DCC has decidable type-checking, dependent types, and uses affine logic. Every primitive object is well-typed, and function types are quite legible. DCC achieves decidable type-checking by eagerly β-reducing all inferred types to canonical, normal form, and by defininining definitional equality as β-equality ($A \equiv B \coloneqq A =_\beta B$). DCC achieves legible types by encoding Π-types as a first-class combinator which indexes an arbitrary environment denoting a context, which is shuffled down a nested Π chain during application.

Definition 17. *If $A, B \in E$, then $A \equiv B \coloneqq A =_\beta B$ 16.*

Definition 18. *The inference rule for Π is as such:*

$$\frac{B : A \to \mathit{Sort}\ u \qquad C : (x : A) \to (y : Bx) \to \mathit{Sort}\ v}{(\Pi\ B\ C) : A \to \mathit{Sort}\ \max(u, v)}\ \textit{PI}$$

Definition 19. *Arrows, written in the meta-level as $A \to B$ are identified by the rule:*

$$\frac{A \to^!_\beta A' \qquad B \to^!_\beta B'}{A' \to B' \coloneqq ((\Pi\ (`\mathbb{1}\ A)\ (`\mathbb{1}\ (`A\ B)))*)}\ \textit{Arrow Def.}$$

We use $(\to\ A\ B)$ to explicitly denote the DCC expansion of $A \to B$.

Definition 20. *We always use the notation $(\Sigma\ A\ B)$ to refer to the DCC object denoting a Σ-type having a first component x of type A and a second component of type (Bx): We make use of a function $h_{\max} : E \to \mathbb{N}$ which calculates the maximum hole value contained in a term tree to determine that B is an arrow from A to some sort, rather than the syntactic expansion, but only in this particular case.*

$$\frac{A : \mathit{Sort}\ u \quad B : ((\Pi\ B\ C)env) \quad A \equiv (Benv) \quad (C\ env\ hole(h_{\max}(((\Pi\ B\ C)\ env)) + 1)) \to^!_\beta \mathit{Sort}\ v}{(\Sigma\ A\ B) : \mathit{Sort}\ \max(u, v)}\ \textit{Sigma}$$

Definition 21. *Application is type-checked as such:*

$$\frac{f : ((\Pi\ B\ C)\ env) \qquad x : A \qquad (B\ env) \equiv A \qquad (C\ env\ x) \to^!_\beta \tau}{(fx) : \tau}\ \textit{App}$$

Note that the inferred type of (fx) is in normal form.

To see an example of how application type-checks, we shall enumerate the inference rules, in the order that their reduction rules are defined.

Definition 22. *Constant terms are inferred like such:*

$$\frac{}{\mathit{Sort}\ u : \mathit{Sort}\ u + 1}\ \textit{Sort} \qquad \frac{}{\mathbb{1} : \mathit{Sort}\ 0}\ \textit{Top} \qquad \frac{}{\mathbb{0} : \mathit{Sort}\ 0}\ \textit{Bot} \qquad \frac{}{* : \mathbb{1}}\ \textit{Top intro} \qquad \frac{}{\mathbb{1}_2 : \mathbb{2}}\ \textit{True} \qquad \frac{}{\mathbb{0}_2 : \mathbb{2}}\ \textit{False}$$

Definition 23. *The identity, quotation, and composition are as such:*

$$\frac{A : \mathit{Sort}\ u \qquad A \to^!_\beta A'}{(Id\ A) : (\to\ A'\ A')}\ \textit{Id} \qquad \frac{x : A \qquad B : \mathit{Sort}\ v \qquad B \to^!_\beta B'}{(`B\ x) : (\to\ B'\ A)}\ \textit{Quot}$$

$$\frac{g : A \to B \qquad f : ((\Pi\ (`\mathbb{1}\ B)\ (`\mathbb{1}\ C))*) \qquad g \to^!_\beta g'}{(\circ\ f\ g) : ((\Pi\ (`\mathbb{1}\ A)\ (`\mathbb{1}\ (\circ\ C\ g)))*)}\ \textit{Comp}$$

In the familiar $(x : A) \to Bx$ notation, the inference rule for composition is written as:

$$\frac{g : A \to B \qquad f : (x : B) \to Cx \qquad g \to^!_\beta g'}{(\circ\ f\ g) : (x : A) \to C(gx)}\ \textit{Comp}$$

Example 5. $((Id\ \mathbb{1})*) : \mathbb{1}$.

This is since $\mathbb{1} : Sort\ 0$ *implies* $(Id\ \mathbb{1}) : (\to\ \ \mathbb{1}\ \mathbb{1})$*, and* $(\to\ \ \mathbb{1}\ \mathbb{1}) = ((\Pi\ (`\mathbb{1}\ \mathbb{1})\ (`\mathbb{1}\ (`\mathbb{1}\ \mathbb{1})))*)$ *and* $* : \mathbb{1}$ *and* $((`\mathbb{1}\ \mathbb{1})*) \equiv \mathbb{1}$*, and* $((`\mathbb{1}\ \mathbb{1})*) \to^!_\beta \mathbb{1}$.

From henceforth, we will use the notation $(x : A) \to Bx$ to denote dependent products for clarity. Consult the Appendix A for the expanded inference rules.

Definition 24. *Projections and currying are as such:*

$$\textit{Fst}\ \frac{B : A \to Sort\ v \qquad B \to^!_\beta B'}{(Fst\ B) : (\Sigma\ A\ B') \to A} \qquad \textit{Snd}\ \frac{B : A \to Sort\ v \qquad B \to^!_\beta B'}{(Snd\ B) : (x : (\Sigma\ A\ B')) \to B'((Fst\ B')x)}$$

$$\textit{Curry}\ \frac{A : Sort\ u \quad A \to^!_\beta A' \quad B : A' \to Sort\ v \quad B \to^!_\beta B' \quad x : (x : (\Sigma\ A'\ B')) \to Cx}{(\Lambda\ A\ B\ x) : (x : A') \to (y : B'x) \to ((\Lambda\ A'\ B'\ C)xy)}$$

Definition 25. *Evaluation is as such:*

$$\textit{Eval}\ \frac{A : Sort\ u \quad A \to^!_\beta A' \quad B : A' \to Sort\ v \quad B \to^!_\beta B' \quad x : (x : A') \to (y : B'x) \to Cxy}{(Eval\ A\ B\ x) : (x : (\Sigma\ A\ B)) \to ((Eval\ A'\ B'\ C)x)}$$

Definition 26. *Exchange is as such:*

$$\textit{Swap}\ \frac{A : Sort\ u \quad B : Sort\ v \quad A \to^!_\beta A' \quad B \to^!_\beta B'}{(Swap\ A\ B) : (\Sigma\ A'\ (`A'\ B')) \to (\Sigma\ B'\ (`B'\ A'))}$$

Definition 27. *The basic operations on boxed terms are as such:*

$$\textit{Unbox}\ \frac{A : Sort\ u \quad A \to^!_\beta A'}{(*\ A) : (!\ A') \to A'} \qquad \textit{Map}\ \frac{x : A \to Sort\ v}{(!<>\ \ x) : (!\ A') \to Sort\ v} \qquad \textit{Fmap}\ \frac{x : (x : A) \to Bx}{(\$<>\ \ x) : (x : (!\ A)) \to ((!<>\ \ B)x)}$$

Definition 28. *Currying on boxes is as such:*

$$\textit{Curry'}\ \frac{A : Sort\ u \quad A \to^!_\beta A' \quad B : A' \to Sort\ v \quad B \to^!_\beta B' \quad x : (x : (!\ (\Sigma\ A'\ B'))) \to Cx}{(!\Lambda\ A\ B\ x) : (x : (!\ A')) \to (y : ((!<>\ \ B')x)) \to ((!\Lambda\ A\ B\ C)xy)}$$

Definition 29. *Pairs are as such:*

$$\textit{Pair}\ \frac{C \to^!_\beta C' \quad x : (x : A) \to Bx \quad y : (x : A) \to (y : Bx) \to C'xy \quad c : (x : A) \to (y : Bx) \to Sort\ s}{(\langle,\rangle C\ x\ y) : (x : (!\ A)) \to ((\Sigma'\ B\ C)x)}$$

Definition 30. *Equality formers are as such:*

$$\textit{Eq'}\ \frac{B : A \to Sort\ v \quad B \to^!_\beta B'}{(='\ \ B) : (x : A) \to (y : B'x) \to (z : B'x) \to Sort\ 0} \qquad \textit{Eql}\ \frac{B : A \to Sort\ v \quad B \to^!_\beta B'}{(Eql\ B) : (x : (!\ A)) \to (y : ((!<>\ \ B')x)) \to Sort\ 0}$$

Definition 31. *The affine* Σ*-former is as such:*

$$\textit{Sigma'}\ \frac{B : A \to Sort\ v \quad B \to^!_\beta B' \quad C : (x : A) \to (y : B'x) \to Sort\ s \quad C \to^!_\beta C'}{(\Sigma'\ B\ C) : (x : (!\ A)) \to Sort\ \max(v, s)}$$

Definition 32. *The recursors for* $\mathbb{2}$ *and* $\mathbb{0}$ *are as such:*

$$\textit{Bool Rec}\ \frac{B : A \to \mathbb{2} \to Sort\ v \quad B \to^!_\beta B' \quad x : (x : A) \to (y : \mathbb{2}) \to B'xy \quad y : (x : A) \to (y : \mathbb{2}) \to B'xy}{(\mathbb{2}.Rec\ x\ y) : (x : A) \to (y : \mathbb{2}) \to B'xy}$$

$$\textit{Bot Rec}\ \frac{B : A \to \mathbb{0} \to Sort\ v \quad B \to^!_\beta B'}{(\mathbb{0}.Rec\ B) : (x : A) \to (y : \mathbb{0}) \to B'xy}$$

Definition 33. *Σ-couples are as such:*

$$\frac{x : A \quad B : A \to \mathit{Sort}\ v \quad B \to^!_\beta B' \quad (B'x) \to^!_\beta\ !\tau_2 \quad y : \tau_2}{(, B\ x\ y) : (\Sigma\ A\ B')}\ \textsc{Couple}$$

Definition 34. *Finally, affine terms are as such:*

$$\frac{A : \mathit{Sort}\ u}{(!\ A) : \mathit{Sort}\ u}\ \textsc{Bang} \qquad \frac{x : A}{(\&\ x) : (!\ A)}\ \textsc{Box}$$

5.1 Type-Checking is Decidable

Our inference rules do not assume the existence of a catch-all conversion rule. Instead, inferred types are reduced to normal form in each rule individually, and definitional equality of terms $A, B \in E$ is checked by comparing the normal-form reducts given by *reduce*. This enables decidable type-checking. We have implemented a type-checker on a deep embedding of DCC in Lean 4, and have shown that it is judgmentally complete, making type-checking decidable.

The source code of the type-checker is available in the Appendix A. Of note, judgments carry computational content and live in **Type**, rather than **Prop**: Judge : $E \to E \to Type$. The type-checker is total and equipped with soundness: infer : $(e : E) \to$ Except (TypeError e)$(\Sigma'(t : E), \text{Judge}\ e\ t)$. Judgmental completeness is provable from the fact that all the inference rules are mutually exclusive, and there is no catch-all conversion.

Theorem 9. *Judgmental completeness: If $e, \tau \in E$ and $h \vdash e : \tau$, then infer$e = ok\langle\tau, h\rangle$.*

Proof. By induction on the typing derivation h. Cases are mutually exclusive, so the proof is reduced to checking that each branch implies that its corresponding branch in infer is *ok*. In each case, if a definitional equality is supposed by h, such that $h_{eq} \vdash A \equiv B$ where $A, B \in E$, then there exists $C \in E$ and $h_{eq,1} \vdash A \to^!_\beta C$ and $h_{eq,2} \vdash B \to^!_\beta C$. So, by completeness of β-reduction with respect to normal forms 6, $reduceA = \langle C, h_{eq,1}\rangle = \langle C, h_{eq,2}\rangle = reduceB$. Hence, $A \equiv B$ in $inferA$ as well, in each case.
✔ [Formalized: `Expr.Judge.complete`] □

Theorem 10. *If $A, \tau_1, \tau_2 \in E$ and $e : \tau_1$ and $e : \tau_2$, then $\tau_1 = \tau_2$.*

Proof. If $h_1 \vdash e : \tau_1$ and $h_2 \vdash e : \tau_2$, then by judgmental completeness 9, $infere = ok\langle\tau_1, h_1\rangle = infere = ok\langle\tau_2, h_2\rangle$, so $\tau_1 = \tau_2$.
✔ [Formalized: `Expr.Judge.unique`] □

Theorem 11. *Inferred types are normal: If $e, \tau \in E$ and $h \vdash e : \tau$, then τ is a β-normal form.*

Proof. We proceed by induction on the typing derivation h:

$e \in E_c$: If e is a constant, then τ is one of $\{\text{Sort}\ u + 1, \text{Sort}\ 0, \mathbb{0}, \mathbb{1}, \mathbb{2}\}$, in which case τ is β-normal.

$e = (c\ x_0 \ldots x_n)$, where $c \in C$ and $\{x_0 \ldots x_n\} \subset E$: If e is a combinator, then any point mentioned in its type τ has been reduced to β-normal form, or was inferred from the type of a point, in which case it is in β-normal form by the inductive hypothesis. And, if e is a combinator, it is either a function, a type former, or a boxed term:

- e **is a function:** Then τ is of the form $((\Pi\ B\ C)x)$, where B, C, and x are in β-normal form due to the above reasoning.
- e **is a type former:** Then τ is of the form Sort u, in which case it is in β-normal form.
- e **is a boxed term:** Then τ is of the form $(!\ A)$, where A is inferred by a point of e, in which case A is in β-normal form by the inductive hypothesis. Hence, τ is in β-normal form.

$e = (fx)$, where $f \in E$ and $x \in E$: If e is an application, then $f : ((\Pi\ B\ C)\ env$ and $x : A$. So, τ is the β-normal reduct of $C\ env\ x$ (per 21), and it is in β-normal form.

✔ [Formalized: `Expr.Judge.is_norm`] □

5.2 Subject-Reduction, Progress, and Consistency

The subject-reduction and progress theorems hold in DCC. We show that if $A : \tau$ and $A \to_\beta B$, then $B : \tau$. We define a set of canonical values $\mathrm{Val} \subset E$ syntactically and show that if $A : \tau$, then there must exist B such that $A \to_\beta B$ or $A \in \mathrm{Val}$. Using these results, we show that $\mathbb{0}$ is uninhabited.

Theorem 12. *Preservation: If $A, B, \tau \in E$, $ht \vdash A : \tau$, and $hr \vdash A \to_\beta B$, then $B : \tau$.*

Proof. We proceed by induction on the reduction relation hr:

A is of the form $(f\,x)$ and B is of the form $(f'\,x)$ where $hr_f \vdash f \to_\beta f'$: Suppose $f : ((\Pi\, B\, C)\ env)$ and $x : A$ and $(Benv) \equiv A$. Then, the inductive hypothesis gives by hr_f that $f' : ((\Pi\ B\ C)\ env)$. Then, by the application rule, $(f'x) : \tau$.

A is of the form $(f\,x)$ and B is of the form $(f\,x')$ where $hr_x \vdash x \to_\beta x'$: Suppose $f : ((\Pi\, B\, C)\ env)$ and $x : A$ and $(Benv) \equiv A$ and $C\ env\ x \to_\beta^! \tau$. Then, the inductive hypothesis gives by hr_x that $x' : A$, and by confluence and argument congruence, $(C\ env\ x') \to_\beta^! \tau$, so $(f x') : \tau$.

A is of the form $(c\ x_0 \ldots x_n)$ where $c \in C$ and $\{x_0 \ldots x_n\} \subset E$: If $(c\ x_0 \ldots x_n) : \tau$, then each point x_i for all $i < n$ is asserted by ht to have a type τ_i and / or may be treated as a value in τ. In case $x_i \to_\beta x_i'$, then its corresponding value in τ is already in β-normal form by 11. And if $x_i : \tau_i$, then by the inductive hypothesis, $x_i' : \tau_i$, so $(c\ x_0' \ldots x_n') : \tau$ by the applicable inference rule.

A is of the form $(f\,x)$ and hr is given by a head step: In case hr is given by a head reduction step, we proceed by cases. In each head-reduction case, the typing relation is inverted, and the innermost definitional equalities yielding τ are destructed given τ is β-normal and unique. Then, the judgment $B : \tau$ is constructed from the available assumptions.

✔ [Formalized: `Expr.Judge.preservation`] □

Definition 35. *The set of canonical values is defined to be the union of the set of constant terms E_c, the set of combinators whose points are values $\{(c\ x_0 \ldots x_n) \mid c \in C, \{x_0 \ldots x_n\} \subset Val\}$, application of the reflexivity combinator, and all possible partially applied terms:*

$$E_c \subset Val \qquad \{(c\ x_0 \ldots x_n) \mid c \in C, \{x_0 \ldots x_n\} \subset Val\} \subset Val$$
$$\{((refl\ B)(\&\ x)) \mid B, x \in Val\} \subset Val \qquad \{((Eql\ B)(\&\ x)) \mid B, x \in Val\} \subset Val$$
$$\{((refl\ B)(\&\ x)(\&\ y)) \mid B, x, y \in Val\} \subset Val \qquad \{((='\ B)x) \mid B, x \in Val\} \subset Val$$
$$\{((='\ B)xy) \mid B, x, y \in Val\} \subset Val \qquad \{((\Pi\ B\ C)x) \mid B, C, x \in Val\} \subset Val$$
$$\{((\Lambda\ A\ B\ x)y) \mid A, B, x, y \in Val\} \subset Val \qquad \{((!\Lambda\ A\ B\ x)(\&\ y)) \mid A, B, x, y \in Val\} \subset Val$$
$$\{((\mathbb{2}.Rec\ B\ x\ y)z) \mid B, x, y, z \in Val\} \subset Val \qquad \{((\mathbb{0}.Rec\ B)x) \mid B, x \in Val\} \subset Val$$

Note that $((\mathbb{0}.Rec\ B)x)$ denotes partial application of the recursor for $\mathbb{0}$ with an environment, not an inhabitant of $\mathbb{0}$.

Lemma 6. *If $h \vdash (f\,x) \in Val$, then $f \in Val$.*

Proof. By case analysis on h.
✔ [Formalized: `Expr.Value.congrFun`] □

Lemma 7. *If $h \vdash (f\,x) \in Val$, then $x \in Val$.*

Proof. By case analysis on h.
✔ [Formalized: `Expr.Value.congrArg`] □

Lemma 8. *If $h \vdash e \in Val$, then e is a β-normal form.*

Proof. By induction on h.
✔ [Formalized: `Expr.Value.is_norm`] □

Progress may be established by noticing that a partially applied combinator is always a value, unless it expects a boxed argument of the form $(\&\ x)$, a coupled term of the form $(, B\ x\ y)$, or an inhabitant of $\mathbb{2}$. The only circumstance where a reduction gets "stuck" is a function is applied the appropriate number of arguments, but they are all in β-normal form and are not the appropriate shape. By inductive reasoning, an argument x to a function must either be a value, or must make a step. Hence, it suffices to prove inversion lemmas relating functions over bang arguments, Σ-arguments, and booleans to their respective value arguments in case x cannot make a step to show that $f x_0 \ldots x_n \in \mathrm{Val}$.

Lemma 9. *If $x \in Val$ and $h \vdash x : (!\ A)$, then there exists $x_0 \in A$ such that $x = (\&\ x_0)$.*

Proof. We proceed by case analysis on the value x. In any case where x is not of the form $(\&\ x_0)$, the judgment h may be inverted to show that $x : \tau$ and τ is not of the form $(!\ A)$. For example, combinators which are functions never have a type of the form $(!\ A)$. For the type-formers, $x : \mathrm{Sort}\ u$ for some $u \in \mathbb{N}$, but $\mathrm{Sort}\ u \neq (\ A)$ for all $A \in E$. In the case of the canonical partially applied combinators, h is similarly inverted to show that $x : \tau$ and τ is of the form $((\Pi\ B\ C)\ env)$, but $((\Pi\ B\ C)\ env) \neq (!\ A)$ for all $A \in E$.
✔ [Formalized: `Expr.Value.inv_box`] □

Lemma 10. *If $h \vdash ((\Pi\ B\ C)x) : \tau$, then there exists $u \in \mathbb{N}$ such that $\tau = Sort\ u$.*

Proof. By inversion on h.

✔ [Formalized: `Expr.Judge.inv_pi`] □

Lemma 11. *If $h \vdash ((Eql\ B)(\&\ x)(\&\ y)) : \tau$, then $\tau = Sort\ 0$.*

Proof. By inversion on h.
✔ [Formalized: `Expr.Judge.inv_eql`] □

Lemma 12. *If $h \vdash ((refl\ B)(\&\ x)(\&\ y)) : \tau$, and $(Bx) \to_\beta^! B'$ and $y \to_\beta^! y'$, then $\tau = ((=\ B')y'y')$.*

Proof. By inversion on h.
✔ [Formalized: `Expr.Judge.inv_refl`] □

Lemma 13. *If $ht \vdash e : \mathbb{0}$ and $hv \vdash e \in Val$, then $\bot$.*

Proof. We proceed by case analysis on ht. The proof can be reduced to checking that no partial application of a combinator can inhabit $\mathbb{0}$, since no inference rule could yield $\mathbb{0}$ in the consequent besides application. In case $e = (fx)$ where $f, x \in E$, we observe that (fx) is a value, so it must be a partial application of a function, or a full application of $refl$. In case f is a function value whose arity is greater than one, inversion shows that there must exist $B, C, env \in E$ such that $(fx) : ((\Pi\ B\ C)\ env)$, but $((\Pi\ B\ C)\ env) \neq \mathbb{0}$ for all $B, C, env \in E$. In case (fx) is a full application of $refl$, we know by 6 that there must exist $B, y \in E$ such that $(fx) : ((=\ B)yy)$, but $((=\ B)yy) \neq \mathbb{0}$ for all $B, y \in E$, so $\not\in : \mathbb{0}$ and thus a contradiction is derived.
✔ [Formalized: `Expr.Judge.elim_bot_value`] □

Lemma 14. *If $ht \vdash e : (A\ B)$ and $hv \vdash e \in Val$, then there exist $x_0, y_0 \in E$ such that $e = (,B\ x\ y)$.*

Proof. By case analysis on hv and parallel inversion on ht.
✔ [Formalized: `Expr.Value.inv_sigma`] □

Lemma 15. *If $ht \vdash e : \mathbb{2}$ and $hv \vdash e \in Val$, then $e = \mathbb{1}_\mathbb{2}$ or $e = \mathbb{0}_\mathbb{2}$.*

Proof. By case analysis on hv and parallel inversion on ht.
✔ [Formalized: `Expr.Value.inv_bool`] □

Using these facts about inversion, we prove progress.

Theorem 13. *If $h \vdash e : \tau$, then $e \in Val$ or there exists $e' \in E$ such that $e \to_\beta e'$.*

Proof. The proof proceeds by induction on h. If $e \in E_c$, then $e \in \mathrm{Val}$. If $e = (c\ x_0 \ldots x_n)$ where $c \in C$ and $\{x_0 \ldots x_n\} \subset E$, then h demonstrates that there exist τ_i such that $x_i : \tau_i$ for all $i < n$, and so each point of e is either a value, or can make a step. In case all points of e are values, $e \in \mathrm{Val}$. If any point can make a step, then so can e by the point rule. If e is of the form (fx) where $f, x \in E$, then h gives that $f : ((\Pi\ B\ C)\ env)$ for some $B, C, env \in E$, and $x : A$ and $(Benv) \equiv A$. Furthermore, by the inductive hypothesis, both f and x must be values or able to make a step. In case either f or x can make a step, function congruence and argument congruent respectively give that e can make a step. In case both f and x are values, we perform case analysis on f, and show that (fx) is either a partial application of a combinator, a full application of $refl$, or, by the established inversion lemmas, x is of the right shape such that (fx) can make a head-step reduction.
✔ [Formalized: `Expr.Judge.progress`] □

Theorem 14. *Consistency: There does not exist $e \in E$ such that $e : \mathbb{0}$.*

Proof. This fact is due to the inversion lemma on values inhabiting $\mathbb{0}$, well-founded recursion by decreasing size under $\rightarrow_\beta$, progress, and preservation. Suppose $e \in E$ such that $h \vdash e : \mathbb{0}$. Then, by progress 13, $e \in \text{Val}$ or there exists $e' \in E$ such that $e \rightarrow_\beta e'$. If $e \in \text{Val}$, then by the value bottom inhabitance inversion lemma (13), a contradiction is derived. If $e \rightarrow_\beta e'$, then $size(e') < size(e)$ by strong normalization 1. And, by preservation 12, $e' : \mathbb{0}$. So, by well-founded recursion, a contradiction is derived.
✔ [Formalized: `Expr.Judge.elim_bot`] □

We note that the following theorems are proven as computable definitions, rather tham theorems in **Prop**: subject-reduction, progress, and consistency. Furthermore, the progress theorem gives a computable choice of $e' \in E$ such that $e \rightarrow_\beta e'$ in case e) is not a value, rather than proving its existence existentially.

6 Computable Equality Reflection

The extensional equality reflection theorem holds in DCC. This is a result of decidable type-checking and strong normalization avoiding a generalized conversion constructor. Since inferred types are always in β-normal form, and since $refl$ is the canonical inhabitant of equality, $e : (= A\ x\ y)$ iff $x \equiv y$. This fact is provable from progress, preservation, and well-founded recursion by strong normalization.

Lemma 16. *If $ht \vdash e : (= A\ x\ y)$ and $hv \vdash e \in \text{Val}$, then $x \equiv y$.*

Proof. The proof proceeds by case analysis on the typing derivation ht. No inference rule can yield $(= A\ x\ y)$ in the consequent aside from in application. Hence, e must be of the form $(f x)$ where $f, x \in E$ and $f, x \in \text{Val}$. We note that $(= A\ x\ y)$ is not a function type. Hence, $(f x)$ cannot be a partial application of a function, but $(f x)$ is not a term with points, nor is it a constant value. So, $(f x)$ must be of the form $((refl\ B)(\&\ x)(\&\ y))$. We demonstrate this fact by inverting the typing derivation ht in each case where f is a value. We demonstrate that in every case except where $(f x) = ((refl\ B)(\&\ x)(\&\ y))$, f is a function whose arity is greater than one, and so $(f x)$ must be typed at τ where τ is of the form $((\Pi\ B\ C)\ env)$ where $B, C, env \in E$, but $((\Pi\ B\ C)\ env) \neq (= A\ x\ y)$ for all $B, C, env, A, x, y \in E$.

So, if $(f x) = ((refl\ B)(\&\ x)(\&\ y))$ for some $B, x, y \in E$, then by inversion on reflexivity values , $((refl\ B)(\&\ x)(\&\ y))$ must be typed at τ where $(B x) \rightarrow_\beta^! B'$ and $y \rightarrow_\beta^! y'$, and $\tau = ((=\ B')y'y')$. But by uniqueness of judgments , $((=\ B')y'y') = ((=\ A\ x\ y)$, so $y' = x$ and $y' = y$. Hence, $x \equiv y$ by reflexivity.
✔ [Formalized: `Expr.Value.elim_eq`] □

Theorem 15. *If $e : (= A\ x\ y)$, then $x \equiv y$.*

Proof. This fact is provable by progress, preservation, well-founded recursion with respect to the reduct of e, in case it is not a value. By progress 13, $e \in \text{Val}$ or there exists $e' \in E$ such that $e \rightarrow_\beta e'$. If $e \in \text{Val}$, then by the above 16, $x \equiv y$. If $e \rightarrow_\beta e'$, then $size(e') < size(e)$ by strong normalization 1. And, by preservation 12, $e' : (= A\ x\ y)$. So, by well-founded recursion, $x \equiv y$.
✔ [Formalized: `Expr.Judge.elim_eq`] □

We note that equality reflection is a computable definition in DCC, rather than a theorem in **Prop**. That is, it has computational content and can be used outside of proofs.

7 Compiling Lean 4 to DCC

The subset of Lean 4 that maps to the base Calculus of Constructions without equality can be compiled to DCC. We note that working examples are somewhat limited, since we refrained from equipping DCC with η-reduction, as it is outside of the scope of this paper and not necessary to demonstrate the fundamental theorems. However, in DCC with η-reduction, the dependent K combinator written as a Lean term can be compiled to DCC. We do not propose a method of encoding inductive types in DCC, as it is out of scope.

Compilation is quite straightforward. Contexts introduced by binders during compilation are represented as Σ or bang ! terms and the program is always initialized at the top-level with the empty context $(!\ \mathbb{1})$. Debruijn-indexed variables are compiled to composed projections, λ-binders are compiled to $!\Lambda$-terms, and $\forall$-terms are compiled to Π-terms. The only complication is DCC's affine restriction:

Lean terms are preprocessed to annotate code denoting how many times a resource should be banged, and values are boxed appropriately.

We are most concerned with compiling the subset of Lean with $\forall$, λ, sorts, booleans, top and bottom types (and their corresponding introduction rules), Σ-types, and Σ-terms.

Definition 36. *We denote L_c as the set of compilation candidate constant Lean terms (where $\top$ denotes the Lean term $True$, $\bot$ denotes the Lean term $False$, and $*$ denotes the Lean term $True.intro$, $\mathbb{2}$ denotes $Bool$, and $\mathbb{1}_2$ denotes $Bool.true$ and $\mathbb{0}_2$ denotes $Bool.false$):*

$$L_c \coloneqq \{\top, *, \bot, \mathbb{2}, \mathbb{1}_2, \mathbb{0}_2, Sort\ u\}$$

where $u \in \mathbb{N}$ (that is, we only consider concrete universes and not metavariables).

Definition 37. *We denote L as the set of candidate Lean terms:*

$$L_c \subset L \quad \{n \in \mathbb{N} \mid n \text{ is a variable}\} \subset L$$
$$\{(MN) \mid M, N \in L\} \subset L \quad \{\lambda x : A.M \mid A, M \in L\} \subset L$$
$$\{\forall x : A.B \mid A, B \in L\} \subset L \quad \left\{\sum_{x:A} B(x) \mid A, B \in L\right\} \subset L \quad \{\langle x, y\rangle \mid x, y \in L\} \subset L$$
$$\{Shared\ A \mid A \in L\} \subset L \quad \{share\ Ax \mid A, x \in L\} \subset L$$

Definition 38. *We write the set of contexts tracked in compilation and introduced by binders as Ctx:*

$$\mathbb{1} \in Ctx \qquad \{(\Sigma\ A\ B) \in E \mid A \in Ctx, B \in Val\} \subset Ctx \qquad \{(!\ A) \in E \mid A \in Ctx\} \subset Ctx$$

Compilation is distinct at the top level and under binders (λ, $\forall$, and Σ-types and terms). We write $[\![M]\!]_\Gamma \in E$ to denote compilation of the term $M \in L$ under a binder (abstraction). We write $[\![M]\!] \in E$ to denote compilation of the term $M \in L$ at the top-level when it is not under a binder. Variables are abstracted by composed projection and unboxing of banged contexts, or the identity if the context is empty. Constants are abstracted by quoting the context, or mapping the context with ! <> if the constant is shared.

Definition 39. *Variable abstraction is as such:*

$$\frac{}{[\![n]\!]_{\mathbb{1}} \coloneqq (Id\ \mathbb{1})} \qquad \frac{[\![n]\!]_\Gamma = n'}{[\![n]\!]_{(!\ \Gamma)} \coloneqq (\circ\ n'\ (*\ \Gamma))}$$
$$\frac{}{[\![0]\!]_\Gamma \coloneqq (Snd\ \Gamma)} \qquad \frac{[\![n]\!]_\Gamma = n'}{[\![n+1]\!]_{(\Sigma\ \Gamma\ \Delta)} \coloneqq (\circ\ n'\ (Fst\ \Delta))}$$

Definition 40. *Constant abstraction is as such:*

$$\frac{}{[\![\top]\!]_\Gamma \coloneqq (`\Gamma\ \mathbb{1})} \qquad \frac{}{[\![*]\!]_\Gamma \coloneqq (`\Gamma\ *)}$$
$$\frac{}{[\![\bot]\!]_\Gamma \coloneqq (`\Gamma\ \mathbb{0})} \qquad \frac{}{[\![\mathbb{2}]\!]_\Gamma \coloneqq (`\Gamma\ \mathbb{2})}$$
$$\frac{}{[\![\mathbb{1}_2]\!]_\Gamma \coloneqq (`\Gamma\ \mathbb{1}_2)} \qquad \frac{}{[\![\mathbb{0}_2]\!]_\Gamma \coloneqq (`\Gamma\ \mathbb{0}_2)}$$
$$\frac{}{[\![Sort\ u]\!]_\Gamma \coloneqq (`\Gamma\ Sort\ u)}$$

λ-terms are always abstracted with !Λ, since we assume the context is always of the form (! Γ) $\in$ Ctx. $\forall$-abstraction distinguishes between $\forall$-types which depend on the context and those which do not (i.e., if A or B does not depend on the context in $\forall A, B$). In case a $\forall$-type is nondependent, it is abstracted as a constant arrow discarding the context as in 19. Σ-terms also assume the context is shared, since A and B may depend on the context in $\sum_{x:A} B(x)$ and are compiled to a Σ' type former which duplicates the context across the compilations of A and B.

Definition 41. *Binder abstraction is as such:*

$$\frac{[\![A]\!]_\Gamma = A' \quad [\![M]\!]_{(!\ (\Sigma\ \Gamma\ A'))} = M'}{[\![\lambda x : (\mathit{Shared}\ A).M]\!]_{(!\ \Gamma)} \coloneqq (!\Lambda\ \Gamma\ A'\ M')}$$

$$\frac{A \text{ or } B \text{ contain free variables} \quad [\![A]\!]_{(!\ \Gamma)} = A' \quad [\![B]\!]_{(!\ (\Sigma\ \Gamma\ A'))} = B'}{[\![\forall x : A, B]\!]_{(!\ \Gamma)} \coloneqq (\Pi\ A'\ (!\Lambda\ \Gamma\ A'\ B'))}$$

$$\frac{\text{neither } A \text{ nor } B \text{ contain free variables} \quad [\![A]\!]_{(!\ \mathbb{1})} = A' \quad [\![B]\!]_{(!\ \mathbb{1})} = B'}{[\![\forall x : A, B]\!]_\Gamma \coloneqq (`\Gamma\ (\to\ \ A'\ B'))}$$

$$\frac{[\![A]\!]_\Gamma = A' \quad [\![B]\!]_{(\Sigma\ \Gamma\ A')} = B'}{\left[\!\!\left[\sum_{x:A} B(x)\right]\!\!\right]_{(!\ \Gamma)} \coloneqq (\Sigma'\ A'\ B')}$$

where $\to$ *denotes the DCC arrow expansion as in 19.*

Applications and Σ-terms under binders duplicate the context across their constituent components (M and N in $(MN) \in L$ and x and y in $\langle x, y\rangle \in L$). So, they are abstracted with the pair combinator 29, assuming the context is of the form $(!\ \Gamma) \in \mathrm{Ctx}$. Applications $(MN) \in L$ are abstracted by composing the evaluation combinator on M' with duplication of the context $x : \Gamma$ via the pair combinator to yield $(\mathit{Eval}\ A\ B\ M')((, B_2\ x\ (N'x))) \to_\beta (M'x(N'x))$.

Definition 42. *Values are abstracted as such:*

$$\frac{\begin{array}{c}[\![x]\!]_\Gamma = x', x' : ((\Pi\ B_1\ C_1)x_1) \quad [\![y]\!]_\Gamma = y', y' : ((\Pi\ B_1\ C_1)x_1) \\ (\circ\ y'\ (\mathit{Fst}\ (C_1x_1))) : (x : A) \to (y : Bx) \to Cxy\end{array}}{[\![\langle x, y\rangle_{(!\ \gamma)} \coloneqq (\langle,\rangle C_3\ x'\ (\Lambda\ (B_1x_1)\ (C_1x_1)\ (\circ\ y'\ (\mathit{Fst}\ (C_1x_1)))))]\!]}$$

$$\frac{\begin{array}{c}[\![M]\!]_\Gamma = M', M' : ((\Pi\ B_1\ C_1)x_1) \quad [\![N]\!]_\Gamma = N', N' : ((\Pi\ B_2\ C_2)x_2) \\ (\circ\ N'\ (\mathit{Fst}\ (`\Gamma\ \Gamma))) : (x : A) \to (y : Bx) \to Cxy\end{array}}{[\![(MN)]\!]_{(!\ \Gamma)} \coloneqq (\circ\ (\mathit{Eval}\ (B_1x_1)\ (C_2x_2)\ M')\ (\Lambda\ \Gamma\ (`\Gamma\ \Gamma)\ (\circ\ N'\ (\mathit{Fst}\ (`\Gamma\ \Gamma)))))]\!]}$$

$$\frac{[\![x]\!]_\Gamma \coloneqq x'}{[\![\mathit{Shared}\ A]\!]_{(!\ \Gamma)} \coloneqq (!<>\ \ x')} \qquad \frac{[\![x]\!]_\Gamma \coloneqq x'}{[\![\mathit{share}\ Ax]\!]_{(!\ \Gamma)} \coloneqq (\$<>\ \ x')}$$

Compilation at the top-level differs from abstraction in that the context is empty. So, all top-level binders' binder types are compiled recursively without a context. Also, the topmost λ-binder is always applied the empty context $(\&\ *) : (!\ \mathbb{1})$. Finally, all user-supplied top-level values are automatically shared based on the computed maximum context duplication depth (this is done as a preprocessing step by marking values with share).

Definition 43. *Constants are compiled as in 40, except the produced DCC terms are not quoted:*

$$\frac{}{[\![\top]\!] \coloneqq \mathbb{1}} \qquad \frac{}{[\![*]\!] \coloneqq *}$$

$$\frac{}{[\![\bot]\!] \coloneqq \mathbb{0}} \qquad \frac{}{[\![\mathbb{2}]\!] \coloneqq \mathbb{2}}$$

$$\frac{}{[\![\mathbb{1}_2]\!] \coloneqq \mathbb{1}_2} \qquad \frac{}{[\![\mathbb{0}_2]\!] \coloneqq \mathbb{0}_2}$$

$$\frac{}{[\![\mathit{Sort}\ u]\!] \coloneqq \mathit{Sort}\ u}$$

Definition 44. *Shared values are compiled as such:*

$$\frac{}{[\![\mathit{Shared}\ A]\!] \coloneqq (!\ [\![A]\!])} \qquad \frac{}{[\![\mathit{share}\ Ax]\!] \coloneqq (\&\ [\![x]\!])}$$

Definition 45. *Binders are compiled as such:*

$$\frac{[\![A]\!]_{(!\ \mathbb{1})} = A' \quad [\![M]\!]_{(!\ (\Sigma\ !\ A'))} = M'}{[\![\lambda x : (Shared\ A).M]\!] \coloneqq ((!\Lambda\ \mathbb{1}\ A'\ M')(\&\ *))}$$

$$\frac{[\![A]\!] = A' \quad [\![B]\!]_{(!\ (\Sigma\ \mathbb{1}\ A'))} = B'}{[\![\forall x : (Shared\ A), B]\!] \coloneqq ((\Pi\ (`\mathbb{1}\ A')\ (`\mathbb{1}\ ((!\Lambda\ \mathbb{1}\ A'\ B')(\&\ *))))*)}$$

$$\frac{[\![A]\!] = A' \quad [\![B]\!]_{(!\ (\Sigma\ \mathbb{1}\ A'))} = B'}{\left[\!\!\left[\sum_{x:Shared\ A} B(x)\right]\!\!\right] \coloneqq (\Sigma\ A'\ ((!\Lambda\ \mathbb{1}\ A'\ B')(\&\ \mathbb{1})))}$$

Definition 46. *Application and Σ-terms are compiled as such:*

$$\frac{}{[\![\langle x, y\rangle]\!] \coloneqq [\![\langle x, y\rangle]\!]_{(!\ \mathbb{1})}}$$

$$\frac{}{[\![(MN)]\!] \coloneqq ([\![M]\!][\![N]\!])}$$

8 Examples

Several examples of type-checking are available in the DCC repository. We have omitted the details here, since the expressions and their inferred types are quite long and not suitable for this medium. We note that DCC with η-reduction is capable of compiling and type-checking an application of the dependent K combinator written as the lean term:

```
def K_example : Expr :=
  dcc_compile' ((fun (α : Sort 3) (β : α → Sort 2) (x : α) (_y : β x) => x) (Sort 2) (fun x
  ↪  => x) (Sort 1) (Sort 0))
```

✔ [Formalized: `Dcc.Compile.Examples`]

9 Comparison to SK Combinators

Previous attempts by others at dependent combinatory type theory [2] used the SK combinator calculus. Our approach differs in that it is based on an extension of the simply-typed categorical combinators to dependent types. We believe our success in establishing dependent combinatory type theory is due to the one-to-one correspondence between Curien's categorical combinators [5] and cartesian-closed categories. Our approach also differs from prior ones in that we adapted the categorical combinators to affine logic. As we have demonstrated, this combination leads to the desirable property of computable extensional equality reflection, alongside having decidable type-checking.

10 Limitations

We note that the type theory presented in this paper does not have η-reduction, and due to its affine nature, we have not succeeded in recovering W-types. There has been work [7] on establishing inductive types in linear logic, and they may be possible through a least fixedpoint operator. However, they are out of the scope of this paper. Furthermore, we note that due to our eager normalization strategy in judgments, proofs of equalities always have a type of the form $(=\ A\ x\ x)$. However, theorems about equality such as equality being symmetric are still theoretically provable, since type conversion occurs when type-checking application **??**.

11 Conclusion

Our formalization gives an extension of the simply-typed categorical combinators [5, p. 220] to a dependently-typed, affine setting. To this date, our presentation is the most complete development

in dependent combinatory logic: we have a working type-checker written in Lean 4 with decidable type-checking, a procedure for compiling a subset of Lean 4 to that type theory, all objects are well-typed, there are no contexts and no variables, and preservation, progress, consistency, and strong normalization hold, while our type theory is similar in expressivity to the Calculus of Constructions with rsepect to the λ-cube. We summarize by emphasizing two properties of this type theory: by extending the categorical combinators to dependent types, we get a simple, but powerful combinatory dependent type theory which feelsl categorically canonical; and by restricting that type theory to affine logic, we gain computable extensional equality reflection as a definition, rather than a theorem in **Prop**, while having decidable type-checking (a property which is known to be close to impossible to achieve).

A Supplementary Materials

- **Lean 4 formalization repository:**
  https://github.com/lexzaiello/dcc
- **Formalized Reduction rules:**
  https://github.com/lexzaiello/DCC/blob/master/Dcc/Red/Rel.lean#L14
- **Type-checker implementation:**
  https://github.com/lexzaiello/DCC/blob/master/Dcc/Judge3.lean
- **Compiler implementation:**
  https://github.com/lexzaiello/DCC/blob/master/Dcc/Compile/Core.lean

References

[1] Thorsten Altenkirch et al. "Combinatory logic and lambda calculus are equal, algebraically". In: *8th International Conference on Formal Structures for Computation and Deduction (FSCD 2023)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik. 2023, pp. 24–1.

[2] Thorsten Altenkirch et al. "The Münchhausen Method in Type Theory". In: *28th International Conference on Types for Proofs and Programs (TYPES 2022)*. Ed. by Delia Kesner and Pierre-Marie Pédrot. Vol. 269. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, 10:1–10:20. ISBN: 978-3-95977-285-3. DOI: 10.4230/LIPIcs.TYPES.2022.10. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.TYPES.2022.10.

[3] Henk Barendregt. "Introduction to generalized type systems". In: *Journal of Functional Programming* 1.2 (1991), pp. 125–154. DOI: 10.1017/S0956796800020025.

[4] G. Cousineau, P-L. Curien, and M. Mauny. "The categorical abstract machine". In: *Functional Programming Languages and Computer Architecture*. Ed. by Jean-Pierre Jouannaud. Berlin, Heidelberg: Springer Berlin Heidelberg, 1985, pp. 50–64. ISBN: 978-3-540-39677-2.

[5] P.-L. Curien. "Categorical combinators". In: *Information and Control* 69.1 (1986), pp. 188–254. ISSN: 0019-9958. DOI: https://doi.org/10.1016/S0019-9958(86)80047-X. URL: https://www.sciencedirect.com/science/article/pii/S001999588680047X.

[6] Nicolaas Govert De Bruijn. "Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem". In: *Indagationes mathematicae (proceedings)*. Vol. 75. 5. Elsevier. 1972, pp. 381–392.

[7] Qiancheng Fu and Hongwei Xi. "A calculus of inductive linear constructions". In: *Proceedings of the 8th ACM SIGPLAN International Workshop on Type-Driven Development*. 2023, pp. 1–13.

[8] Jean-Yves Girard. "Linear logic". In: *Theoretical computer science* 50.1 (1987), pp. 1–101.

[9] Jussi Ketonen and Richard Weyhrauch. "A decidable fragment of predicate calculus". In: *Theoretical computer science* 32.3 (1984), pp. 297–307.

[10] Alexey P Kopylov. "Decidability of linear affine logic". In: *Information and Computation* 164.1 (2001), pp. 173–198.

[11] Joachim Lambek and Philip J Scott. *Introduction to higher-order categorical logic*. Vol. 7. Cambridge University Press, 1988.

[12] nLab authors. *Cartesian closed category*. https://ncatlab.org. Accessed: August 13, 2026. URL: https://ncatlab.org/nlab/show/cartesian+closed+category.